

Student **Management** System

A Comprehensive C# Desktop Solution

Powered by .NET WinForms & Persistent Storage

Development Team & Sections

محمد علاء حلمي

Section: 38



محمد يوسف ابراهيم

Section: 39



محمود رضا كامل

Section: 38



محمد عبدالله الحميد

Section: 37



محمد هاني شحاته

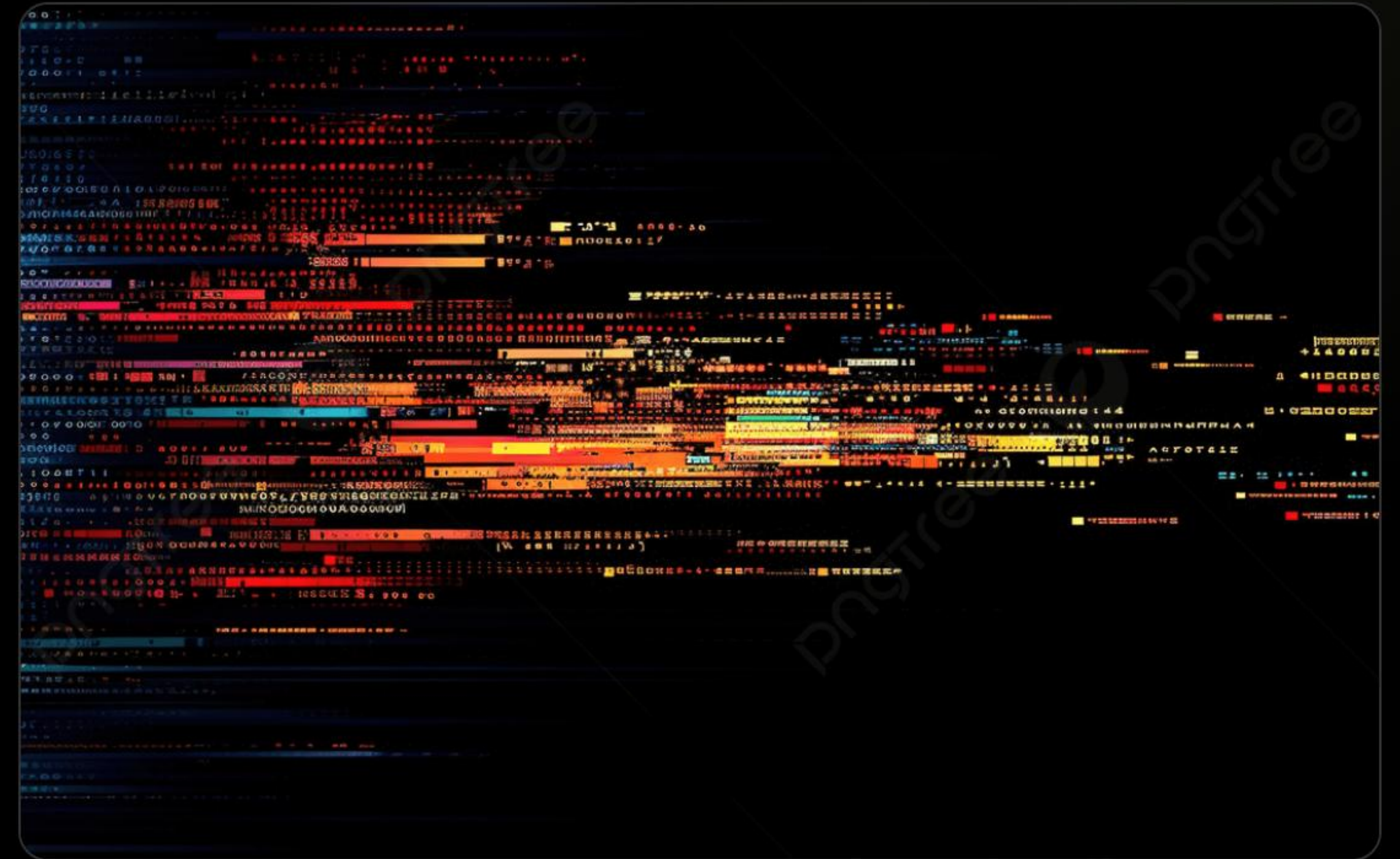
Section: 39

Project Overview

Digital transformation in education requires robust systems to handle student data efficiently.

Our project automates the administrative workflow by replacing manual record-keeping with a high-performance desktop application.

- ⚡ High Efficiency
- 🛡️ Data Integrity
- 💾 Reliable Persistence



Core Capabilities



Registration

Capture Name, Age, and Level with validation.



Modification

Instantly update existing records through grid.



Deletion

Permanent record removal with confirmation.



Data Persistence: Automatic saving to flat-file storage (students.txt).

Technical Stack



Language

C# (C-Sharp) with .NET Framework integration.



Interface

Windows Forms for a native desktop experience.



Persistence

File-based I/O using System.IO namespace.

System Data Model

Property	Type	Description
ID	Integer	Auto-incrementing unique identifier.
Name	String	Full name of the student.
Age	Integer	Numerical age with input validation.
Level	Enum	Academic year (First to Fourth).

System Initialization

The application loads existing data from "students.txt" on startup.

```
if (File.Exists("students.txt")) {
    var lines = File.ReadAllLines("students.txt");
    foreach (var line in lines) {
        var data = line.Split(',');
        students.Add(new Student() {
            ID = int.Parse(data[0]),
            Name = data[1],
            Age = int.Parse(data[2]),
            Level = (Level)Enum.Parse(...)
        });
    }
}
```

```
late(WorkCalculatorLaborRequest obj, bool? IsRecalc = false, List<HistogramConfigEntity> histogramOrex = null,
    result.Labor.Count - 1; iCount++)

    dCostOfSalary = 0, dOverTimeCoastWeek = 0;
    0, dOverTimeCoastSat = 0, dAdditionalHourSat = 0, dOverTimeCoastSun = 0;
    }, dDissidium = 0, dInsalubrity = 0, dHazardousness = 0, dFixedSocialChargeCost = 0;
    am = 0, dFoodBasked = 0, dFoodVoucher = 0, dManagerTraning = 0, dTurnOver = 0, dHealthPlan = 0, dLifeInsurance
    TotalEpi = 0, dTotalThirdService = 0, dTotalDirectLabor = 0;
    Materials = 0, dTotalTools = 0, dTotalEquip = 0, dTotalStay = 0, dTotalTravel = 0;
    = 0, dTotalToolkit = 0;
    WorkTransport = 0;

    les = 0;

    lltKit = null;
    resultProducts = new List<ProductToolkitResponse>();
    EPI = null;
    stay = null;
    ransport = null;
    Equip = null;
    Tool = null;
    Materials = null;

    t = new HistogramConfigRequest()

    Count].ID_LABOR,
    AMETER

    _histogramConfigRepository.ListHistogramConfig(request);
    amOrex == null ? await _histogramConfigRepository.ListHistogramConfig(request) : histogramOrex.Where(x => x.ID_

    CostOfSalaryHour(result, result.Labor[iCount]);

    (OD == 2) //Mensal
    llcCostOfSalary(result, dSalaryValueHour, result.Labor[iCount], resultHistogram, resultParameter.ToList());
    :CostOfSalary(result.Labor[iCount]);

    llcCostOfSalaryDay(result, dSalaryValueHour, result.Labor[iCount], resultHistogram, resultParameter.ToList());
    :CostOfSalaryDay(result, result.Labor[iCount], resultParameter.ToList());

    rs(result, result.Labor[iCount], resultParameter.ToList());
    llcOvertimeCoastWeek(result, dSalaryValueHour, resultParameter.ToList(), result.Labor[iCount], resultHistogram);
    llcAdditionalWeekHourCost(dOverTimeCoastWeek, resultParameter.ToList());
    :OvertimeCoastSat(result, dSalaryValueHour, resultParameter.ToList(), result.Labor[iCount]);
    llcAdditionalHourSat(dOverTimeCoastSat, resultParameter.ToList());
    :OvertimeCoastSun(result, dSalaryValueHour, resultParameter.ToList(), result.Labor[iCount]);
```


Operational Impact

0.5s

Average Retrieval Time

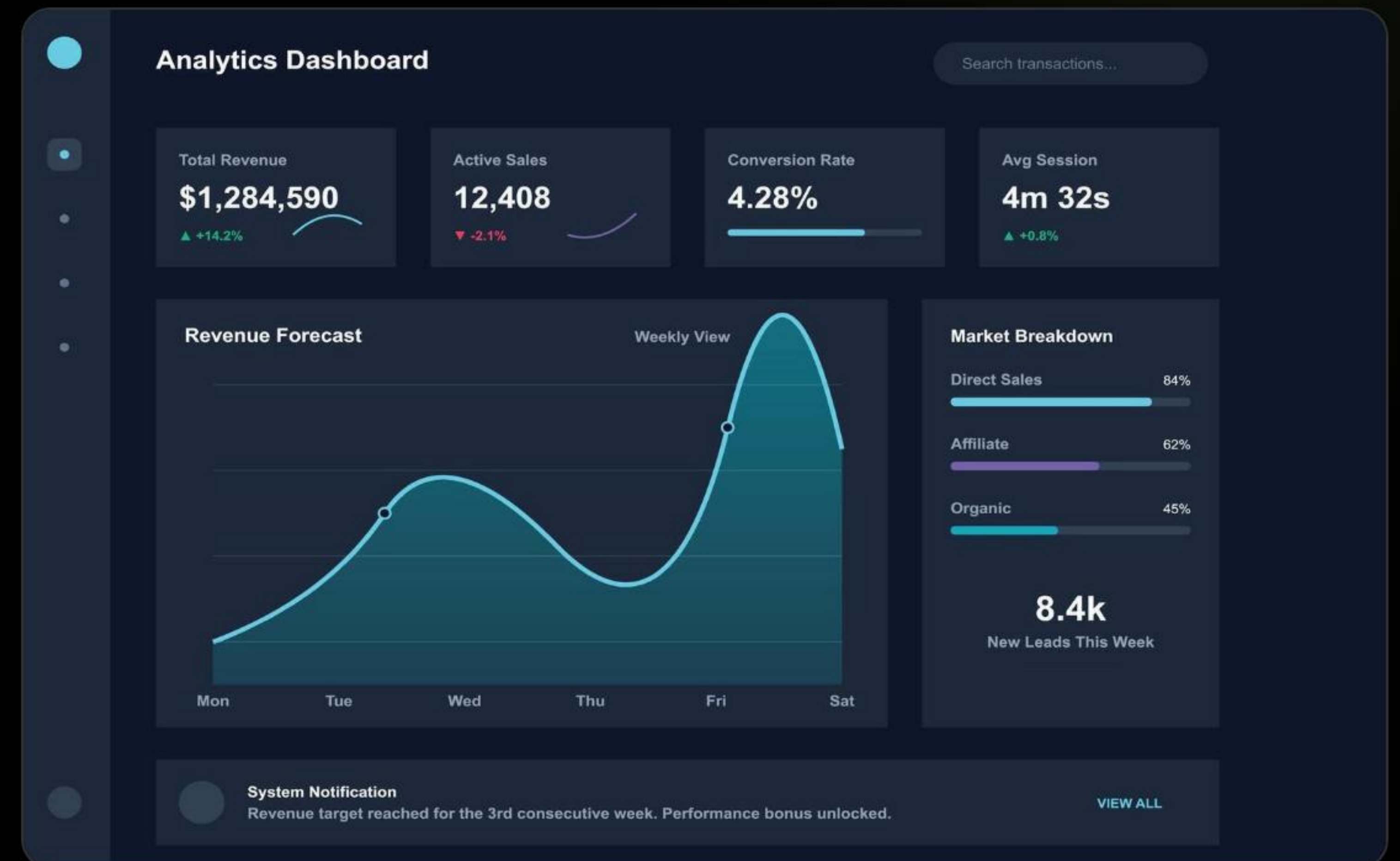
Efficiency Redefined

By implementing a local List collection and DataGridView binding, the system provides near-instantaneous feedback.

Persistence operations happen in the background, ensuring no latency during data entry.

Robust **CRUD** Operations

```
private void btnAdd_Click_1(...) {  
    try {  
        Student s = new Student() {  
            ID = currentID++,  
            Name = txtName.Text,  
            Age = int.Parse(txtAge.Text),  
            Level = (Level)comboLevel.SelectedItem  
        };  
        students.Add(s);  
        SaveToFile();  
    } catch { MessageBox.Show("Error"); }  
}
```



Intuitive UI Design



Selection Binding

Clicking a row auto-fills input fields for editing.



Auto-Fill Grid

DataGridView adjusts column width automatically.



User Validation

Instant visual feedback through *MessageBox*.

Future Roadmap



Any Questions?

Thank you for your attention!

Student Management Project v1.0 | 2024

Image Sources



Thumbnail

for

https://png.pngtree.com/background/20250215/original/pngtree-abstract-modern-tech-of-programming-code-screen-developer-picture-image_15504908.jpg

Source: [pngtree.com](https://png.pngtree.com)



pngtree.com

<https://preview.redd.it/programming-war-crimes-v0-8m3vamk3xawdl.jpg?width=1771&format=pjpg&auto=webp&s=8dee24e4e25390d5acb490206a6435f031fc08d2>

Source: www.reddit.com



https://static.vecteezy.com/system/resources/previews/077/050/554/non_2x/analytics-dashboard-ui-template-with-dark-theme-data-visualization-charts-and-graphs-for-web-site-or-app-vector.jpg

Source: www.vecteezy.com
